

Low-Code API Orchestration Platform

Sample Configurations

Name: Revanth Kumar Dompaka

Roll No: S20230010073

B.Tech in CSE

Indian Institute of Information Technology, Sri City

This document contains the sample MongoDB configurations used by the platform. These are the actual documents stored in the three collections — Vendor, ApiConfig, and ExecutionLog. They can be seeded using `npm run seed`.

1 Vendor Configurations

Vendor documents are stored in the Vendor collection. Each document represents a third-party service the platform can call. The `responseSchema` field is used by the AI agent to generate correct field mappings automatically.

1.1 VendorA — PAN Verification

```
{
  "name": "VendorA",
  "description": "PAN verification vendor - expects panNumber
    and fullName",
  "baseUrl": "http://localhost:5000",
  "endpoint": "/vendors/vendorA/verify-pan",
  "method": "POST",
  "headers": {},
  "authentication": {
    "type": "NONE",
    "value": ""
  },
  "timeout": 5000,
  "retry": {
    "count": 3,
    "delay": 1000
  },
  "responseSchema": {
```

```

        "verificationStatus": "string - VALID or INVALID",
        "score": "number - confidence score 0 to 100",
        "panNumber": "string",
        "fullName": "string"
    },
    "isActive": true
}

```

1.2 VendorB — PAN Verification (Alternative Format)

```

{
    "name": "VendorB",
    "description": "PAN verification vendor (different format) -
        expects documentId",
    "baseUrl": "http://localhost:5000",
    "endpoint": "/vendors/vendorB/verify-pan",
    "method": "POST",
    "headers": {},
    "authentication": {
        "type": "NONE",
        "value": ""
    },
    "timeout": 5000,
    "retry": {
        "count": 3,
        "delay": 1000
    },
    "responseSchema": {
        "result": "string - SUCCESS or FAILURE",
        "confidence": "number - confidence score 0 to 100",
        "documentId": "string"
    },
    "isActive": true
}

```

1.3 VendorC — Aadhaar Verification

```

{
    "name": "VendorC",

```

```

    "description": "Aadhaar verification vendor - expects
        aadhaarNumber and name",
    "baseUrl": "http://localhost:5000",
    "endpoint": "/vendors/vendorC/verify-aadhaar",
    "method": "POST",
    "headers": {},
    "authentication": {
        "type": "NONE",
        "value": ""
    },
    "timeout": 5000,
    "retry": {
        "count": 3,
        "delay": 1000
    },
    "responseSchema": {
        "status": "string - VERIFIED or FAILED",
        "aadhaarNumber": "string",
        "name": "string",
        "verifiedAt": "string - ISO date"
    },
    "isActive": true
}

```

1.4 VendorD — OCR Extraction

```

{
    "name": "VendorD",
    "description": "OCR extraction vendor - expects
        documentBase64",
    "baseUrl": "http://localhost:5000",
    "endpoint": "/vendors/vendorD/ocr-extract",
    "method": "POST",
    "headers": {},
    "authentication": {
        "type": "NONE",
        "value": ""
    },
    "timeout": 10000,
    "retry": {
        "count": 2,

```

```

        "delay": 2000
    },
    "responseSchema": {
        "name": "string - extracted name",
        "dob": "string - extracted date of birth",
        "documentType": "string - extracted document type",
        "documentNumber": "string - extracted document number",
        "confidence": "number - OCR confidence 0 to 100",
        "status": "string - EXTRACTED or FAILED"
    },
    "isActive": true
}

```

1.5 VendorE — Fraud Detection

```

{
    "name": "VendorE",
    "description": "Fraud detection vendor - expects name and
        documentNumber",
    "baseUrl": "http://localhost:5000",
    "endpoint": "/vendors/vendorE/fraud-detect",
    "method": "POST",
    "headers": {},
    "authentication": {
        "type": "NONE",
        "value": ""
    },
    "timeout": 8000,
    "retry": {
        "count": 3,
        "delay": 1000
    },
    "responseSchema": {
        "fraudScore": "number - fraud score 0 to 100",
        "riskLevel": "string - LOW MEDIUM or HIGH",
        "flagged": "boolean - true if flagged for fraud",
        "documentNumber": "string",
        "checkedAt": "string - ISO date"
    },
    "isActive": true
}

```

1.6 VendorF — Face Match

```
{
  "name": "VendorF",
  "description": "Face match vendor - expects selfieBase64 and
    documentBase64",
  "baseUrl": "http://localhost:5000",
  "endpoint": "/vendors/vendorF/face-match",
  "method": "POST",
  "headers": {},
  "authentication": {
    "type": "NONE",
    "value": ""
  },
  "timeout": 8000,
  "retry": {
    "count": 3,
    "delay": 1000
  },
  "responseSchema": {
    "matched": "boolean - true if face matched",
    "similarity": "number - similarity score 0 to 100",
    "status": "string - MATCH or NO_MATCH",
    "checkedAt": "string - ISO date"
  },
  "isActive": true
}
```

2 API Config Configurations

ApiConfig documents are stored in the ApiConfig collection. Each document defines one orchestrated endpoint — its validation rules, workflow steps, field mappings, and conditions.

2.1 Config 1 — Single-Step PAN Verification

Demonstrates: request mapping, response mapping, regex validation.

```
{
  "name": "Verify PAN",
```

```

"description": "validates a PAN number using VendorA. maps
    pan+name to panNumber+fullName",
"endpoint": "/verify-pan",
"method": "POST",
"version": "v1",
"validation": [
    {
        "field": "pan",
        "required": true,
        "type": "string",
        "regex": "^[A-Z]{5}[0-9]{4}[A-Z]{1}$",
        "message": "pan must be in valid format eg.
            ABCDE1234F"
    },
    {
        "field": "name",
        "required": true,
        "type": "string",
        "regex": null,
        "message": "name is required"
    }
],
"workflow": [
    {
        "step": 1,
        "vendor": "<VendorA ObjectId>",
        "requestMapping": {
            "pan": "panNumber",
            "name": "fullName"
        },
        "responseMapping": {
            "verificationStatus": "status",
            "score": "confidence"
        },
        "condition": {
            "field": null,
            "operator": null,
            "value": null
        },
        "parallel": false,
        "onFailure": "STOP"
    }
]

```

```

    }
  ],
  "isActive": true
}

```

2.2 Config 2 — Single-Step Aadhaar Verification

Demonstrates: different validation rules, different vendor, different field mapping.

```

{
  "name": "Verify Aadhaar",
  "description": "validates an aadhaar number using VendorC.
    maps aadhaar+name fields",
  "endpoint": "/verify-aadhaar",
  "method": "POST",
  "version": "v1",
  "validation": [
    {
      "field": "aadhaar",
      "required": true,
      "type": "string",
      "regex": "^[0-9]{12}$",
      "message": "aadhaar must be a 12 digit number"
    },
    {
      "field": "name",
      "required": true,
      "type": "string",
      "regex": null,
      "message": "name is required"
    }
  ],
  "workflow": [
    {
      "step": 1,
      "vendor": "<VendorC ObjectId>",
      "requestMapping": {
        "aadhaar": "aadhaarNumber",
        "name": "name"
      },
      "responseMapping": {
        "status": "verificationStatus",

```

```

        "verifiedAt": "verifiedAt"
      },
      "condition": {
        "field": null,
        "operator": null,
        "value": null
      },
      "parallel": false,
      "onFailure": "STOP"
    }
  ],
  "isActive": true
}

```

2.3 Config 3 — Multi-Step Document Verification Pipeline

Demonstrates: multi-step orchestration, conditional execution, response aggregation across three vendors.

- Step 1 — OCR extraction (VendorD) — always runs
- Step 2 — Fraud detection (VendorE) — runs only if OCR returned EXTRACTED
- Step 3 — Face match (VendorF) — runs only if fraud check returned isFlagged = false

```

{
  "name": "Verify Document",
  "description": "full document verification pipeline - ocr
    then fraud detection then face match",
  "endpoint": "/verify-document",
  "method": "POST",
  "version": "v1",
  "validation": [
    {
      "field": "documentBase64",
      "required": true,
      "type": "string",
      "regex": null,
      "message": "documentBase64 is required"
    },
    {

```



```

        "field": "selfieBase64",
        "required": true,
        "type": "string",
        "regex": null,
        "message": "selfieBase64 is required"
    }
],
"workflow": [
    {
        "step": 1,
        "vendor": "<VendorD ObjectId>",
        "requestMapping": {
            "documentBase64": "documentBase64"
        },
        "responseMapping": {
            "name": "name",
            "documentNumber": "documentNumber",
            "documentType": "documentType",
            "confidence": "ocrConfidence",
            "status": "ocrStatus"
        },
        "condition": {
            "field": null,
            "operator": null,
            "value": null
        },
        "parallel": false,
        "onFailure": "STOP"
    },
    {
        "step": 2,
        "vendor": "<VendorE ObjectId>",
        "requestMapping": {
            "name": "name",
            "documentNumber": "documentNumber",
            "documentType": "documentType"
        },
        "responseMapping": {
            "fraudScore": "fraudScore",
            "riskLevel": "riskLevel",
            "flagged": "isFlagged"
        }
    }
]

```

```

    },
    "condition": {
        "field": "ocrStatus",
        "operator": "eq",
        "value": "EXTRACTED"
    },
    "parallel": false,
    "onFailure": "STOP"
},
{
    "step": 3,
    "vendor": "<VendorF ObjectId>",
    "requestMapping": {
        "selfieBase64": "selfieBase64",
        "documentBase64": "documentBase64"
    },
    "responseMapping": {
        "matched": "faceMatched",
        "similarity": "similarity",
        "status": "faceMatchStatus"
    },
    "condition": {
        "field": "isFlagged",
        "operator": "eq",
        "value": false
    },
    "parallel": false,
    "onFailure": "CONTINUE"
}
],
"isActive": true
}

```

2.4 Config 4 — Aadhaar and Voter ID (Manually Created)

Demonstrates: adding a new vendor (VendorG) and creating a multi-step workflow manually via the CRUD API with zero code changes.

```

{
    "name": "Verify Aadhaar and Voter ID",
    "description": "verifies Aadhaar using VendorC and if
        verified checks Voter ID using VendorG",

```

```

"endpoint": "/verify-aadhaar-voter",
"method": "POST",
"version": "v1",
"validation": [
  {
    "field": "aadhaar",
    "required": true,
    "type": "string",
    "regex": "^[0-9]{12}$",
    "message": "aadhaar must be a 12 digit number"
  },
  {
    "field": "name",
    "required": true,
    "type": "string",
    "regex": null,
    "message": "name is required"
  },
  {
    "field": "voterId",
    "required": true,
    "type": "string",
    "regex": "^[A-Z]{3}[0-9]{7}$",
    "message": "voterId must be in valid format eg.
      ABC1234567"
  }
],
"workflow": [
  {
    "step": 1,
    "vendor": "<VendorC ObjectId>",
    "requestMapping": {
      "aadhaar": "aadhaarNumber",
      "name": "name"
    },
    "responseMapping": {
      "status": "aadhaarStatus",
      "verifiedAt": "aadhaarVerifiedAt"
    },
    "condition": {
      "field": null,

```

```

        "operator": null,
        "value": null
    },
    "parallel": false,
    "onFailure": "STOP"
},
{
    "step": 2,
    "vendor": "<VendorG ObjectId>",
    "requestMapping": {
        "voterId": "voterId",
        "name": "name"
    },
    "responseMapping": {
        "voterStatus": "voterStatus",
        "constituency": "constituency",
        "verifiedAt": "voterVerifiedAt"
    },
    "condition": {
        "field": "aadhaarStatus",
        "operator": "eq",
        "value": "VERIFIED"
    },
    "parallel": false,
    "onFailure": "STOP"
}
],
"isActive": true
}

```

2.5 Config 5 — PAN and Fraud Detection (AI Generated)

Demonstrates: the AI agent generating a complete multi-step config from a single natural language prompt. This config was not written manually — it was generated by Gemini 2.5 Flash.

Prompt used:

```

"Create an API that verifies PAN using VendorA and if successful
runs fraud detection using VendorE"

```

Generated config:

```

{
    "name": "Verify PAN and Detect Fraud",

```

```

"description": "validates a PAN using VendorA and runs fraud
  detection using VendorE if successful",
"endpoint": "/verify-pan-fraud",
"method": "POST",
"version": "v1",
"validation": [
  {
    "field": "pan",
    "required": true,
    "type": "string",
    "regex": "^[A-Z]{5}[0-9]{4}[A-Z]{1}$",
    "message": "pan must be in valid format"
  },
  {
    "field": "name",
    "required": true,
    "type": "string",
    "regex": null,
    "message": "name is required"
  }
],
"workflow": [
  {
    "step": 1,
    "vendor": "<VendorA ObjectId>",
    "requestMapping": {
      "pan": "panNumber",
      "name": "fullName"
    },
    "responseMapping": {
      "verificationStatus": "panVerificationStatus",
      "score": "panConfidenceScore"
    },
    "condition": {
      "field": null,
      "operator": null,
      "value": null
    },
    "parallel": false,
    "onFailure": "STOP"
  },

```

```

    {
      "step": 2,
      "vendor": "<VendorE ObjectId>",
      "requestMapping": {
        "name": "name",
        "pan": "documentNumber"
      },
      "responseMapping": {
        "fraudScore": "fraudScore",
        "riskLevel": "riskLevel",
        "flagged": "isFlaggedForFraud"
      },
      "condition": {
        "field": "panVerificationStatus",
        "operator": "eq",
        "value": "VALID"
      },
      "parallel": false,
      "onFailure": "CONTINUE"
    }
  ],
  "isActive": true
}

```

3 Execution Log Sample

Every request is logged to the ExecutionLog collection automatically. Below is a sample log from a successful /verify-pan execution showing the complete step-by-step trace.

```

{
  "requestId": "req_574f0667-0175-40f4-b56f-26d82f423bee",
  "apiConfigId": "<ApiConfig ObjectId>",
  "endpoint": "/verify-pan",
  "method": "POST",
  "status": "SUCCESS",
  "originalRequest": {
    "pan": "ABCDE1234F",
    "name": "Revanth Kumar"
  },
  "finalResponse": {
    "status": "VALID",

```

```

        "confidence": 98
    },
    "steps": [
        {
            "step": 1,
            "vendor": "<VendorA ObjectId>",
            "status": "SUCCESS",
            "requestPayload": {
                "panNumber": "ABCDE1234F",
                "fullName": "Revanth Kumar"
            },
            "responsePayload": {
                "verificationStatus": "VALID",
                "score": 98,
                "panNumber": "ABCDE1234F",
                "fullName": "Revanth Kumar"
            },
            "retryAttempts": 0,
            "startedAt": "2026-07-04T11:22:06.440Z",
            "completedAt": "2026-07-04T11:22:06.740Z",
            "duration": 300
        }
    ],
    "startedAt": "2026-07-04T11:22:06.430Z",
    "completedAt": "2026-07-04T11:22:06.744Z",
    "totalDuration": 314,
    "error": null,
    "createdAt": "2026-07-04T11:22:06.745Z"
}

```

What this log tells you:

- The original client request is stored — pan and name
- Step 1 mapped the request to panNumber and fullName before calling VendorA
- VendorA responded with verificationStatus: VALID and score: 98
- Zero retry attempts — VendorA responded on the first call
- Total end-to-end duration was 314ms
- The final response the client received was status: VALID and confidence: 98 — after response mapping

Every execution — success or failure — is logged this way. If a vendor fails and retries, `retryAttempts` increments. If the workflow stops due to a condition not being met, the skipped steps are logged with `status: SKIPPED`. This gives a complete audit trail for every request the platform handles.